

實驗室：Media CDN 的 Service Extensions

來源：<https://codelabs.developers.google.com/service-extensions-mediacdnhl=zh-tw>

步驟數：12

在本程式碼研究室中，您將建構 Media CDN 發布管道，透過 Service Extensions 外掛程式執行自訂程式碼，以完成自訂的 HTTP 驗證。

目錄

1. 簡介
2. 事前準備
3. 設定和需求
4. 建立 Cloud Storage bucket
5. 將測試物件上傳至 Cloud Storage Bucket
6. 設定 Media CDN
7. 為 Service Extensions 設定 Artifact Registry
8. 在 Media CDN 上設定 Service Extensions
9. 驗證 Media CDN 的 Service Extensions Proxy-Wasm 外掛程式
10. 選用：管理 Proxy-Wasm 外掛程式
11. 清理實驗室環境
12. 恭喜！

步驟 1 · 約 0 分鐘

1. 簡介

上次更新時間：2024 年 5 月 1 日

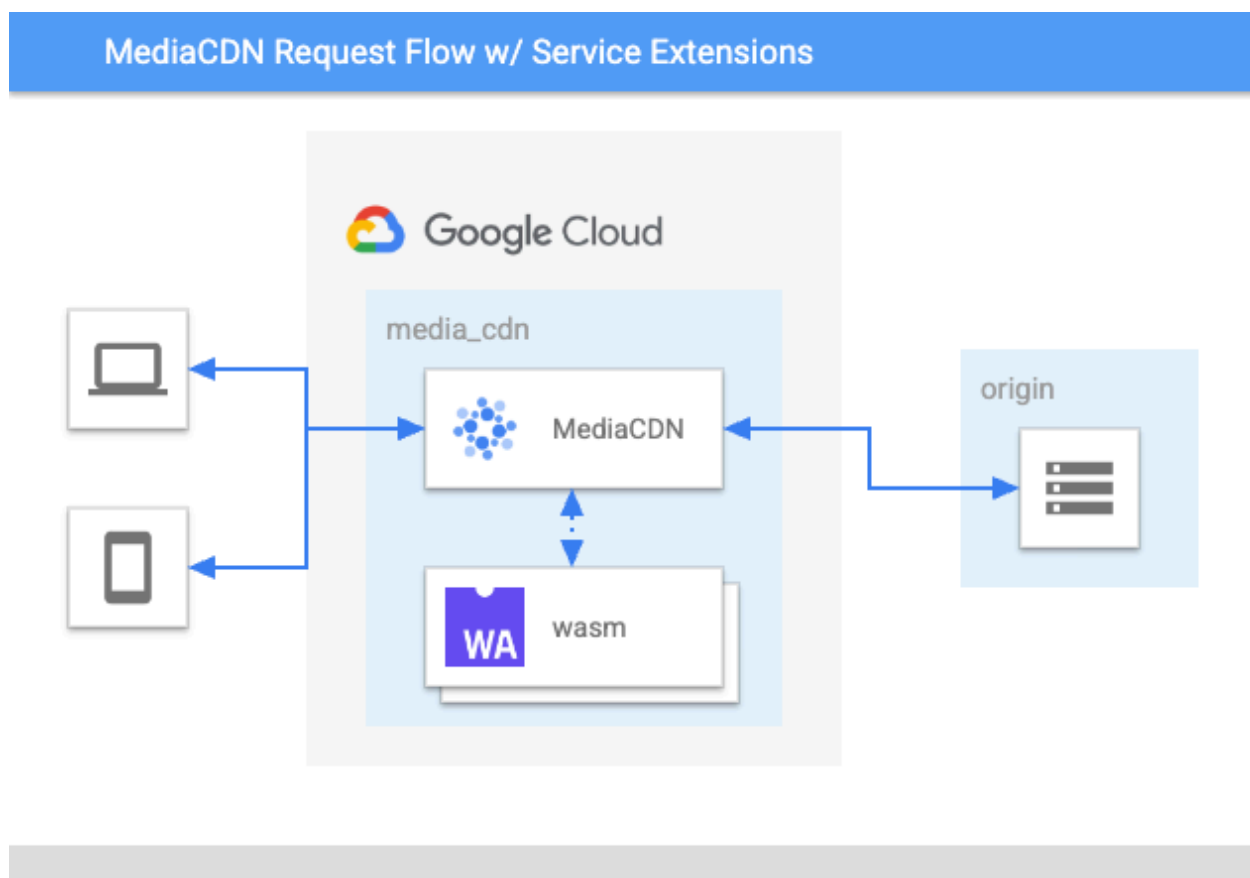
內容傳遞網路 (CDN) 會將經常存取的內容快取在離使用者較近的位置，終止與用戶端較近的連線，重複使用與來源的連線，並採用現代網路通訊協定和自訂項目，藉此提升使用者效能。

Media CDN 是 GCP 用於串流媒體的全球邊緣網路，提供許多內建或「核心」功能。核心功能旨在解決最常見的用途，但您可能也有這組核心功能無法滿足的需求。

Media CDN 的 Service Extensions (有時也稱為 Edge Programmability) 可讓您在邊緣執行自己的程式碼，自訂 Media CDN 的行為。這項功能可解鎖更多用途，包括正規化快取金鑰、自訂權杖驗證和權杖撤銷、額外的自訂記錄檔欄位、A/B 測試和自訂錯誤頁面。

建構項目

在本程式碼研究室中，我們將逐步說明如何部署啟用邊緣運算的 CDN 傳遞環境，其中包含 **Media CDN** (CDN) + **Service Extensions** (Edge Programmability) + **Cloud Storage** (CDN 來源)。



課程內容

- 如何設定 Media CDN，並將 Cloud Storage bucket 設為來源
- 如何建立具有自訂 HTTP 驗證的 Service Extension 外掛程式，並將其與 Media CDN 建立關聯
- 如何驗證 Service Extensions 外掛程式是否正常運作
- (選用) 如何管理 Service Extensions 外掛程式，例如更新、參照、回溯及刪除特定外掛程式版本

軟硬體需求

- 具備基本網路和 HTTP 知識
 - 基本的 Unix/Linux 指令列知識
-

2. 事前準備

要求將 Media CDN 和 Service Extensions 加入許可清單

注意：請確認您的專案已加入 Media CDN 和 Media CDN Service Extensions 的許可清單。如果專案未加入許可清單，您就無法完成本實驗室。

開始使用前，請先確認專案已加入 Media CDN 和 Media CDN Service Extensions 的私人搶先體驗許可清單。

- 如要同時要求存取 Media CDN 和 Media CDN 的 Service Extensions，請與 Google 帳戶團隊聯絡，請他們代您建立 Media CDN 和 Service Extensions 的存取要求

注意：如果您是 Google 員工，且想使用 Media CDN 測試 Service Extensions，請與 Media CDN 和 Service Extensions 的 PM 聯絡，要求存取權。

步驟 3 · 約 2 分鐘

3. 設定和需求

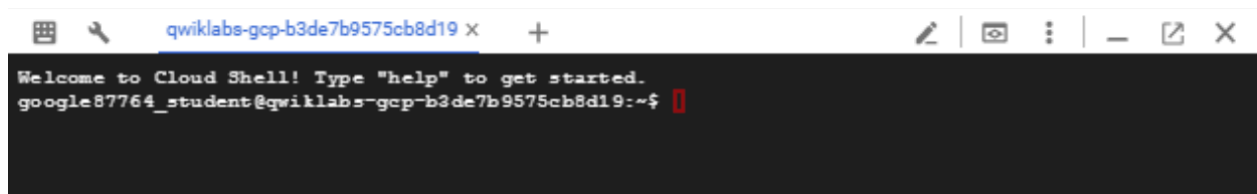
啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

在 GCP 主控台，按一下右上角工具列的 Cloud Shell 圖示：



佈建並連線至環境的作業需要一些時間才能完成。完成後，您應該會看到如下的內容：



這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。您只需要瀏覽器，就能完成本實驗室的所有工作。

事前準備

IAM 角色和存取權

如要建立 Media CDN 和 Artifact Registry 資源，必須具備下列身分與存取權管理 (IAM) 權限：

- roles/networkservices.edgeCacheAdmin
- roles/networkservices.edgeCacheUser
- roles/networkservices.edgeCacheViewer
- roles/artifactregistry.repoAdmin

在 Cloud Shell 中，請確認已設定 **project_id**、**project_num**、**location** 和 **repository** 環境變數。

```
gcloud config list project
gcloud config set project [YOUR-PROJECT-NAME]
PROJECT_ID=[YOUR-PROJECT-NAME]
PROJECT_NUM=[YOUR-PROJECT-NUMBER]
LOCATION=us-central1
REPOSITORY=service-extension-$PROJECT_ID
```

啟用 API

透過下列指令啟用 Media CDN 和 Service Extensions API

```
gcloud services enable networkservices.googleapis.com
gcloud services enable networkactions.googleapis.com
gcloud services enable edgecache.googleapis.com
gcloud services enable artifactregistry.googleapis.com
```

4. 建立 Cloud Storage bucket

Media CDN 內容可來自 Cloud Storage bucket、第三方儲存空間位置，或任何可公開存取的 HTTP(HTTPS) 端點。

在本程式碼研究室中，我們會將內容儲存在 Cloud Storage bucket 中。

注意：Cloud Storage bucket 名稱在全域範圍內都不可重複。為確保沒有衝突，我們會使用 `$PROJECT_ID` 環境變數做為 bucket 名稱的一部分。

我們將使用 **gsutil mb** 指令建立 bucket

```
gsutil mb gs://mediacd-n-bucket-$PROJECT_ID
```

或者，您也可以使用 GUI 建立 Cloud Storage bucket，如下所示：

1. 前往 Google Cloud 控制台中的「Cloud Storage」頁面。
2. 按一下「建立」按鈕。
3. 輸入 bucket 的名稱。 - 也就是「mediacd-n-bucket-\$PROJECT_ID」。
4. 其餘設定保留預設值。
5. 按一下「建立」按鈕。



Cloud Storage



Buckets



Monitoring



Settings



Marketplace



Create a bucket

- **Name your bucket**

Pick a **globally unique**, permanent name. [Naming guidelines](#)

Ex. 'example', 'example_bucket-1', or 'example.com'

Tip: Don't include any sensitive information

✓ **LABELS (OPTIONAL)**

CONTINUE

- **Choose where to store your data**

Location: us (multiple regions in United States)

Location type: Multi-region

- **Choose a storage class for your data**

Default storage class: Standard

- **Choose how to control access to objects**

Public access prevention: On

Access control: Uniform

- **Choose how to protect object data**

Protection tools: None

Data encryption: Google-managed

CREATE

CANCEL

5. 將測試物件上傳至 Cloud Storage Bucket

現在，我們要將物件上傳至 Cloud Storage bucket。

1. 在 Cloud Shell 中建立檔案，然後使用 **gsutil** 上傳至值區

```
echo media-cdn-service-extensions-test > file.txt

gsutil cp file.txt gs://mediacd-bucket-$PROJECT_ID
```

附註：Media CDN 支援將私人 bucket 做為來源。如要允許 Media CDN 存取私人 Cloud Storage 值區的內容，您需要設定 IAM，將物件檢視者存取權授予 Media CDN 服務帳戶。

這個服務帳戶具備下列格式：

service-<PROJECT_NUM>@gcp-sa-mediaedgefill.iam.gserviceaccount.com

2. 授予 Media CDN bucket 存取權

```
gsutil iam ch \
serviceAccount:service-$PROJECT_NUM@gcp-sa-
mediaedgefill.iam.gserviceaccount.com:objectViewer gs://mediacd-bucket-$PROJECT_ID
```

6. 設定 Media CDN

接下來，我們要建立 Media CDN 設定。

每項 Media CDN 設定都包含兩項主要資源：

- **EdgeCacheService**：負責面向用戶端的設定 (TLS、IP 位址)、路由、CDN 設定 (快取模式、存留時間、簽署) 和安全性政策。
- **EdgeCacheOrigin**：負責任何以 HTTP 為基礎的來源設定，以及內容無法使用或無法連線時的重試條件。

設定邊緣快取來源

現在，讓我們建立指向您剛建立的 Cloud Storage 值區的來源。

1. 前往 Google Cloud 控制台的「Media CDN」頁面。
2. 按一下「來源」分頁標籤。
3. 按一下「建立來源」。
4. 輸入「cloud-storage-origin」做為邊緣快取來源的名稱。
5. 在「來源地址」下方：
6. 選擇「選取 Google Cloud Storage bucket」。
7. 瀏覽至名為「mediacd-n-bucket-\$PROJECT_ID」的 Cloud Storage bucket。
8. 按一下「選取」。
9. 其餘設定保留預設值。
10. 按一下「建立來源」。

Network services

Load balancing

Cloud DNS

Cloud CDN

Cloud NAT

Traffic Director

Service Directory

Cloud Domains

Private Service Connect

Media CDN

Marketplace

Release Notes

<|

Create an Edge Cache Origin

Name *

Lowercase, no spaces.

Description

Origin address ?

Select a Google Cloud Storage bucket

Select a bucket *

BROWSE

Specify a FQDN or IP address

Protocol and port

Protocol

HTTP2

Port *

443

Failover Origin ?

Select an Origin

Retry conditions

Retry conditions

CONNECT_FAILURE

Max attempts

Max attempts

1

Timeout

Connect timeout *

5

seconds ?

Max attempts timeout *

15

seconds ?

Response timeout *

30

seconds ?

Read timeout *

15

seconds ?

Labels (Optional) ?

+ ADD LABEL

新建立的 EdgeCacheOrigin 資源會顯示在「來源」頁面中專案的來源清單中。

設定邊緣快取服務

1. 前往 Google Cloud 控制台的「Media CDN」頁面。
2. 按一下「服務」分頁標籤。
3. 按一下「建立服務」。
4. 輸入服務的專屬名稱 (例如「media-cdn」)，然後按一下「下一步」。

Network services

Load balancing

Cloud DNS

Cloud CDN

Cloud NAT

Traffic Director

Service Directory

Cloud Domains

Private Service Connect

Media CDN

Create Edge Cache service

1 Edge Cache service basics

Service name *

Lowercase, no spaces.

Description

Labels (Optional) ?

+ ADD LABEL

NEXT

2 Routing

3 Security & Encryption

CREATE SERVICE

CANCEL

5. 在「Routing」(轉送) 區段中，按一下「ADD HOST RULE」(新增主機規則)。
6. 在「主機」欄位中輸入萬用字元「*」。

Network services

Load balancing

Cloud DNS

Cloud CDN

Cloud NAT

Traffic Director

Service Directory

Cloud Domains

Private Service Connect

Media CDN

Marketplace

Release Notes

Create Edge Cache service

✓ Edge Cache service basics

2 Routing

New host rule

Hosts *

* ✕

Description

Route rules

Priority	Match	Routing action	Origin
No rows to display			

+ ADD ROUTE RULE

DONE

ADD HOST RULE

NEXT

3 Security & Encryption

CREATE SERVICE

CANCEL

- 按一下「新增路徑規則」。
- 在「Priority」(優先順序) 中指定「1」。
- 按一下「新增比對條件」，針對路徑比對選取「前置字元比對」做為比對類型，在路徑比對欄位中指定「/」，然後按一下「完成」。
- 選取「主要動作」下方的「Fetch from an Origin」(從來源擷取)，然後從下拉式清單中選取您設定的來源。

Edit route rule

Priority *

1



Priority is evaluated from 1 (highest) to 999 (lowest).

Description

Match

Edit match condition



Path match

Match type

Prefix match



Path match *

/

☒ Enable case sensitivity for path value

☐ Headers match

☐ Query parameters match

DONE

ADD A MATCH CONDITION

Primary action

☒ Fetch from an origin

☐ URL redirect

Select an origin



▼ ADVANCED CONFIGURATIONS

SAVE

CANCEL

11. 點選「進階設定」，即可展開更多設定選項。

12. 在「Route action」(轉送動作) 部分，按一下「ADD AN ITEM」(新增項目)。然後執行下列操作：

13. 在「類型」部分，選取「CDN 政策」。

14. 在「快取模式」中，選取「強制快取所有內容」。
15. 其餘設定則保留預設值。
16. 按一下 [完成]。
17. 按一下 [儲存]。

Edit route rule

Primary action

☒ Fetch from an origin

Select an origin

☐ URL redirect

Header action

ADD AN ITEM

Route action

New item

Type *

CDN policy

Cache mode

Force cache all

Client TTL

seconds

Default TTL *

3600

seconds

Cache key policy

Options

☐ Include protocol

☐ Exclude query string

☐ Exclude host

Included query parameters

+ ADD QUERY PARAMETER

Excluded query parameters

+ ADD QUERY PARAMETER

SAVE

CANCEL

18. 按一下「建立服務」。

新建立的 EdgeCacheService 資源會顯示在專案的服務清單中。

擷取 MediaCDN IP 位址並進行測試

1. 前往 Google Cloud 控制台的「Media CDN」頁面。
2. 前往 Media CDN
3. 按一下「服務」分頁標籤。

4. 查看服務的「地址」欄。

Media CDN

SERVICES

ORIGINS

KEYSETS

An Edge Cache Service is a public endpoint that makes thousands of global edge locations available for delivering media to your users. [Learn more](#)

Edge Cache Services [+ CREATE SERVICE](#) [DELETE](#)

Filter dummy-service Filter Services

☐	Name ↑	Route Rules	Edge Security Policy	Origins	Keysets	Addresses	Created	Last updated
☐	dummy-service-TF	2		dummy-origin-TF		34.104.33.64 2600:1900:4110:816::	Jun 14, 2023, 7:27:52 PM	Jun 15, 2023, 10:03:40 AM

如要測試服務是否已正確設定為快取內容，請使用 curl 指令列工具發出要求並檢查回應。

```
curl -svo /dev/null "http://MEDIA_CDN_IP_ADDRESS/file.txt"
```

這個指令應該會輸出如下的內容：

```
< HTTP/2 200 OK
...
media-cdn-service-extensions-test
...
```

您已成功建立以 Cloud Storage 為來源的 MediaCDN 部署作業。

7. 為 Service Extensions 設定 Artifact Registry

建立 Service Extensions 前，請先設定 Artifact Registry。Artifact Registry 是 Google Cloud 的通用套件管理工具，用於管理建構構件。服務擴充功能 (Proxy-Wasm) 外掛程式會發布至 Artifact Registry。發布至 Artifact Registry 後，Proxy-Wasm 外掛程式即可部署至 Media CDN 部署作業。

注意：Artifact Registry 存放區名稱在全域範圍內都不可重複。為確保沒有衝突，我們會使用 \$PROJECT_ID 環境變數做為存放區名稱的一部分。

我們將使用 **gcloud artifacts repositories create** 指令建立存放區

```
gcloud artifacts repositories create service-extension-$PROJECT_ID \
  --repository-format=docker \
  --location=$LOCATION \
  --description="Repo for Service Extension" \
  --async
```

您也可以使用 GUI 建立存放區，如下所示：

1. 前往 Google Cloud 控制台的「Artifact Registry」頁面。
2. 按一下「+ 建立存放區」按鈕。
3. 輸入存放區的名稱，例如「service-extension-\$PROJECT_ID」。
4. 格式 - 「Docker」、模式 - 「標準」、位置類型 - 「區域」，然後選取「us-central1 (Iowa)」(us-central1 (愛荷華州))
5. 按一下「建立」按鈕。

 Artifact Registry

Repositories

Settings

← Create repository

Name *

Format

☒ Docker

☐ Maven

☐ npm

☐ Python

☐ Apt

☐ Yum

☐ Kubeflow Pipelines

☐ Go

Mode

☒ Standard

☐ Remote

PREVIEW

☐ Virtual

PREVIEW

Location type

☒ Region

☐ Multi-region

Region *

us-central1 (Iowa)

Description

Labels

+ ADD LABEL

Encryption

This resource is encrypted with a Google-managed key by default. If you need to manage your encryption, you can use a customer-managed key instead. [Learn more](#)

☒ Google-managed encryption key

No configuration required

☐ Customer-managed encryption key (CMEK)

Manage via [Google Cloud Key Management Service](#)

☐ Immutable Tags

PREVIEW

?

CREATE

CANCEL

新建立的 Artifact Registry 存放區資源應會顯示在「存放區」頁面。

建立存放區資源後，請在 Cloud Shell 中執行下列指令，將 Cloud Shell Docker 用戶端設為使用這個存放區推送及提取套件。

```
gcloud auth configure-docker $LOCATION-docker.pkg.dev
```

輸出內容：

...

Adding credentials for: us-central1-docker.pkg.dev

Docker configuration file updated.

8. 在 Media CDN 上設定 Service Extensions

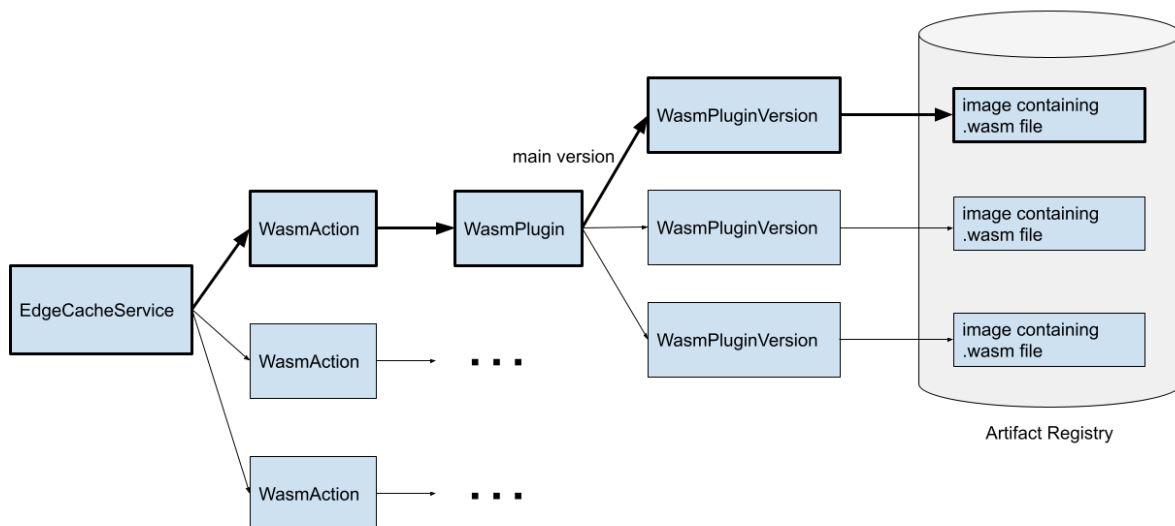
現在，我們將示範如何使用 [Rust 程式設計語言](#)，編寫及建構可部署至 Media CDN 的 Service Extension (Proxy-Wasm) 外掛程式。

在本範例中，我們將建立 Proxy-Wasm 外掛程式，驗證每個 HTTP 要求是否包含值為「secret」的 Authorization 標頭。如果要求不含這個標頭，外掛程式會產生 HTTP 403 Forbidden 回應。

快速複習服務擴充功能：有三項重要資源，分別是 WasmAction、WasmPlugin 和 WasmPluginVersion。

- WasmAction 資源會附加至 Media CDN EdgeCacheService。WasmAction 會參照 WasmPlugin 資源。
- WasmPlugin 資源具有主要版本，對應於目前有效的 WasmPluginVersion。
- WasmPluginVersions 會參照 Artifact Registry 中的容器映像檔。變更 proxy-wasm 外掛程式時，您會建立不同的 WasmPluginVersion。

請參考下圖，進一步瞭解這些資源之間的關係。



編寫及建構服務擴充功能外掛程式

1. 按照 <https://www.rust-lang.org/tools/install> 中的操作說明安裝 Rust 工具鍊。

注意：請務必按照提示操作，安裝 Rust 工具鍊。

如果在執行「rustup」時收到「*-bash: rustup: command not found*」錯誤，請嘗試執行「**source "\$HOME/.cargo/env"**」重新載入 Cloud Shell 中的 PATH 環境。

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

2. 接著，執行下列指令，將 Wasm 支援新增至 Rust 工具鍊：

```
rustup target add wasm32-wasi
```

3. 建立名為 **my-wasm-plugin** 的 Rust 套件：

```
cargo new --lib my-wasm-plugin
```

輸出內容：

```
Created library `my-wasm-plugin` package
```

4. 輸入目錄 **my-wasm-plugin**，您應該會看到 **Cargo.toml** 檔案和 **src** 目錄。

```
cd my-wasm-plugin  
ls
```

輸出內容：

```
Cargo.toml  src
```

5. 接著，編輯 **Cargo.toml** 檔案，設定 Rust 套件。在 Cargo.toml 檔案的 `[dependencies]` 行之後，新增下列內容：

```
proxy-wasm = "0.2"  
log = "0.4"  
  
[lib]  
crate-type = ["cdylib"]  
  
[profile.release]  
lto = true  
opt-level = 3  
codegen-units = 1  
panic = "abort"  
strip = "debuginfo"
```

6. 編輯後，**Cargo.toml** 檔案應如下所示：

```
[package]
name = "my-wasm-plugin"
version = "0.1.0"
edition = "2021"

[dependencies]
proxy-wasm = "0.2"
log = "0.4"

[lib]
crate-type = ["cdylib"]

[profile.release]
lto = true
opt-level = 3
codegen-units = 1
panic = "abort"
strip = "debuginfo"
```

7. 將 `sample_code` 檔案的完整內容複製到 Cloud Shell 中 `src` 目錄的 `lib.rs` 檔案。

附註：`lib.rs` 檔案是原始碼所在的位置。原始碼應以 [Rust 程式設計語言](#) 撰寫。如需更多程式碼範例，請參閱 [network-actions-recipes](#)。

在本程式碼研究室中，我們的程式碼會驗證每個 HTTP 要求是否包含值為「secret」的 Authorization 標頭，如果要求不含這個標頭，就會產生 HTTP 403 Forbidden 回應。

8. 編輯後，`lib.rs` 檔案應如下所示：

```

use log::info;
use proxy_wasm::traits::*;
use proxy_wasm::types::*;

...

struct DemoPlugin;

impl HttpContext for DemoPlugin {
    fn on_http_request_headers(&mut self, _: usize, _: bool) -> Action {
        if self.get_http_request_header("Authorization") == Some(String::from("secret")) {
            info!("Access granted.");
            Action::Continue
        } else {
            self.send_http_response(403, vec![], Some(b"Access forbidden.\n"));
            Action::Pause
        }
    }
}

impl Context for DemoPlugin {}

```

9. 我們已設定 **cargo.toml** 資訊清單檔案，並在 **lib.rs** 檔案中編寫 Proxy-Wasm 程式碼，現在可以建構 Proxy-Wasm 外掛程式。

```
cargo build --release --target wasm32-wasi
```

建構成功完成後，您會看到如下訊息：

```
Finished release [optimized] target(s) in 1.01s
```

我們也來驗證 **target** 目錄，並檢查建立的檔案：

```
ls ./target
```

您會看到如下所示的輸出內容：

```
CACHEDIR.TAG release wasm32-wasi
```

將 Proxy-Wasm 外掛程式發布至 Artifact Registry

現在，我們將 Proxy-Wasm 外掛程式發布至先前建立的 Artifact Registry 存放區，以便部署至 Media CDN。

首先，我們將 Proxy-Wasm 外掛程式封裝到容器映像檔中。

1. 在同一個 **my-wasm-plugin** 目錄中建立名為 **Dockerfile** 的檔案，並加入以下內容：


```
FROM scratch
COPY target/wasm32-wasi/release/my_wasm_plugin.wasm plugin.wasm
```

2. 接著，請建構容器映像檔：

```
docker build --no-cache --platform wasm -t my-wasm-plugin .
```

注意：如果您使用 Apple Silicon (以 ARM 為基礎的架構) 或任何其他非 x86 處理器，請先在 Docker 上啟用 WASM 支援 (<https://docs.docker.com/desktop/wasm/#turn-on-wasm-workloads>)，然後執行下列指令來建構容器映像檔。

(**僅限非 x86 處理器**) 接著，請建構容器映像檔：

```
docker build --no-cache --platform wasm --provenance=false -t my-wasm-plugin .
```

輸出

```
[+] Building 0.2s (5/5) FINISHED                                docker:default
...
```

3. 接著，將 Proxy-Wasm 外掛程式發布或「推送」至 Artifact Registry。我們會使用「prod」標記為容器映像檔加上標記。

注意：Artifact Registry 會使用命名慣例來識別存放區和映像檔。

存放區中 Docker 映像檔的全名格式如下：

LOCATION-docker.pkg.dev/PROJECT-ID/REPOSITORY/IMAGE:TAG

```
docker tag my-wasm-plugin $LOCATION-docker.pkg.dev/$PROJECT_ID/$REPOSITORY/my-wasm-plugin:prod
```

現在，請繼續將加上「prod」標記的容器映像檔推送至存放區。

```
docker push $LOCATION-docker.pkg.dev/$PROJECT_ID/$REPOSITORY/my-wasm-plugin:prod
```

輸出內容：

```
The push refers to repository
...
8564ddd9910a: Pushed
prod: digest: sha256:f3ae4e392eb45393bfd9c200cf8c0c261762f7f39dde5c7cd4b9a8951c6f2812
size: 525
```

注意：請確認您已將 gcloud 設定為與存放區位置相關聯的 Artifact Registry 網域憑證輔助程式。

如果將容器推送至 Artifact Registry 時發生「權限遭拒」錯誤，請返回「為 Service Extensions 設定 Artifact Registry」，然後再次執行 **gcloud auth** 指令。

現在來驗證 Proxy-Wasm 外掛程式的容器映像檔是否已成功推送至 Artifact Registry，您應該會看到類似以下的輸出內容：

```
gcloud artifacts docker images list $LOCATION-  
docker.pkg.dev/$PROJECT_ID/$REPOSITORY/my-wasm-plugin --include-tags
```

輸出內容：

```
Listing items under project  
...  
IMAGE                                DIGEST                                TAGS  CREATE_TIME  
UPDATE_TIME  
<LOCATION>-docker.pkg.dev/.../my-wasm-plugin  sha256:08c12...  prod  2021-11-  
10T23:31:27  2021-11-10T23:31:27
```

將 Proxy-Wasm 外掛程式與 Media CDN 部署作業建立關聯

現在，我們準備將 Proxy-Wasm 外掛程式與 Media CDN 部署作業建立關聯。

Proxy-Wasm 外掛程式會與 EdgeCacheService 資源中的 Media CDN 路徑建立關聯。

注意：建立每個 Proxy-Wasm 資源可能需要一段時間。

請等待系統顯示完成訊息，再繼續執行後續步驟。

1. 首先，我們要為 Proxy-Wasm 外掛程式建立 Wasm 外掛程式資源。

```
gcloud alpha service-extensions wasm-plugins create my-wasm-plugin-resource
```

2. 接著，我們建立 WasmPluginVersion。

```
gcloud alpha service-extensions wasm-plugin-versions create my-version-1 \  
--wasm-plugin=my-wasm-plugin-resource \  
--image="$LOCATION-docker.pkg.dev/$PROJECT_ID/$REPOSITORY/my-wasm-plugin:prod"
```

3. 接著，我們為 Proxy-Wasm 外掛程式指定主要版本。

```
gcloud alpha service-extensions wasm-plugins update my-wasm-plugin-resource \  
--main-version=my-version-1
```

現在，讓我們驗證 Proxy-Wasm 外掛程式是否已成功與 Artifact Registry 存放區中的容器映像檔建立關聯，您應該會看到類似下列的輸出內容：

```
gcloud alpha service-extensions wasm-plugin-versions list --wasm-plugin=my-wasm-plugin-resource
```

輸出內容：

```
NAME      WASM_IMAGE WASM_IMAGE_DIGEST CONFIG_SIZE  CONFIG_IMAGE CONFIG_IMAGE_DIGEST
UPDATE_TIME
c7cfa2 <LOCATION>-docker.pkg.dev/.../my-wasm-plugin@sha256:6d663... ... ...
...
```

4. 接著，我們建立 WasmAction 資源，參照 Wasm 外掛程式資源。

```
gcloud alpha service-extensions wasm-actions create my-wasm-action-resource \
  --wasm-plugin=my-wasm-plugin-resource
```

我們也來驗證 WasmAction 資源是否已成功與 Proxy-Wasm 外掛程式建立關聯，您應該會看到類似以下的輸出內容：

```
gcloud alpha service-extensions wasm-actions list
```

輸出內容：

```
NAME                                     WASMPLUGIN
my-wasm-action-resource
projects/805782461588/locations/global/wasmPlugins/myenvoyfilter-resource
...
```

5. 現在，我們需要匯出 Media CDN EdgeCacheService 的設定：

```
gcloud edge-cache services export media-cdn --destination=my-service.yaml
```

6. 接著開啟 **my-service.yaml** 檔案，並將 **wasmAction** 新增至指定路徑的 **routeAction**，其中會參照先前建立的 WasmPlugin 資源。

```
wasmAction: "my-wasm-action-resource"
```

7. 編輯後，**my-service.yaml** 檔案應如下所示：

```

...

pathMatchers:
  - name: routes
    routeRules:
      - headerAction: {}
        matchRules:
          - prefixMatch: /
            origin: projects/<PROJECT_NUM>/locations/global/edgeCacheOrigins/cloud-storage-origin
            priority: '1'
            routeAction:
              cdnPolicy:
                cacheKeyPolicy: {}
                cacheMode: FORCE_CACHE_ALL
                defaultTtl: 3600s
                signedRequestMode: DISABLED
              wasmAction: "my-wasm-action-resource"
...

```

8. 接著，我們將更新後的設定連同 Proxy-Wasm 設定儲存至 **my-service-with-wasm.yaml** 檔案。

注意：如果將編輯過的 YAML 檔案匯入 Edge-Cache 服務 API 時發生設定驗證錯誤，請仔細檢查 YAML 檔案，確認 **routeAction** 區段中已加入 **wasmAction** 行。請務必只使用空格縮排，YAML 不接受 Tab 鍵。

9. 最後，我們匯入實際工作環境 Media CDN 的更新設定：

```

$ gcloud alpha edge-cache services import media-cdn --source=my-service-with-
wasm.yaml

```

9. 驗證 Media CDN 的 Service Extensions Proxy-Wasm 外掛程式

如要測試服務是否已正確設定為快取內容，請使用 curl 指令列工具發出要求並檢查回應。

注意：這項變更最多可能需要 10 分鐘，才會套用至整個 MediaCDN 網路。在此期間，您可能無法看到 HTTP 403 錯誤。請持續嘗試，直到傳回 403 錯誤為止。

```
curl -svo /dev/null "http://IP_ADDRESS/file.txt"
```

這個指令應該會輸出如下的內容：

```
< HTTP/2 403 Forbidden
...
Access forbidden.
...
```

現在，請再次發出要求，並在授權標頭中加入密鑰值

```
curl -svo /dev/null "http://IP_ADDRESS/file.txt" -H "Authorization: secret"
```

這個指令應該會輸出如下的內容：

```
< HTTP/2 200 OK
...
media-cdn-service-extensions-test
...
```

10. 選用：管理 Proxy-Wasm 外掛程式

更新 Proxy-Wasm 外掛程式

改善或新增 Proxy-Wasm 外掛程式的功能時，您需要將更新後的外掛程式部署至 Media CDN。以下將逐步說明如何部署外掛程式的更新版本。

舉例來說，您可以修改下列程式碼，更新範例外掛程式程式碼，根據其他值評估授權標頭，以進行驗證。

首先，請使用下列程式碼更新 src/lib.rs 來源檔案：

```
use log::{info, warn};
use proxy_wasm::traits::*;
use proxy_wasm::types::*;

...

struct DemoPlugin;

impl HttpContext for DemoPlugin {
    fn on_http_request_headers(&mut self, _: usize, _: bool) -> Action {
        if self.get_http_request_header("Authorization") ==
Some(String::from("another_secret")) {
            info!("Access granted.");
            Action::Continue
        } else {
            warn!("Access forbidden.");
            self.send_http_response(403, vec![], Some(b"Access forbidden.\n"));
            Action::Pause
        }
    }
}

impl Context for DemoPlugin {}
```

接著，請建構、封裝及發布更新後的外掛程式：

```
cargo build --release --target wasm32-wasi
docker build --no-cache --platform wasm -t my-wasm-plugin .
docker tag my-wasm-plugin $LOCATION-docker.pkg.dev/$PROJECT_NUM/$REPOSITORY/my-wasm-
plugin:prod
docker push $LOCATION-docker.pkg.dev/$PROJECT_NUM/$REPOSITORY>/my-wasm-plugin:prod
```

Artifact Registry 中的容器映像檔更新後，我們需要建立新的 WasmPluginVersion，然後更新 WasmPlugin 的「--main-version」，以參照新版本。

```
gcloud alpha service-extensions wasm-plugin-versions create my-version-2 \
  --wasm-plugin=my-wasm-plugin-resource \
  --image="$LOCATION-docker.pkg.dev/$PROJECT_NUM/$REPOSITORY>/my-wasm-plugin:prod"
```

```
gcloud alpha service-extensions wasm-plugins update my-wasm-plugin-resource \
  --main-version=my-version-2
```

您已成功更新要從 Artifact Registry 匯入的容器映像檔版本，並推送至 Media CDN 部署作業。

復原至前一個版本

若要復原外掛程式的先前版本，可以更新 Wasm 外掛程式資源，以參照先前版本。

首先，列出可用版本：

```
gcloud alpha service-extensions wasm-plugin-versions list --wasm-plugin=my-wasm-
plugin-resource
```

您應該會看到以下輸出內容：

NAME	WASM_IMAGE	WASM_IMAGE_DIGEST	CONFIG_SIZE	CONFIG_IMAGE	CONFIG_IMAGE_DIGEST	UPDATE_TIME
c7cfa2	<LOCATION>-docker.pkg.dev/.../my-wasm-plugin@sha256:6d663...	...	...	...	...	...
a2a8ce	<LOCATION>-docker.pkg.dev/.../my-wasm-plugin@sha256:08c12...	...	...	...	...	...

接著，更新 Wasm 外掛程式資源，參照先前的「a2a8ce」版本：

```
$ gcloud alpha service-extensions wasm-plugins update my-wasm-plugin-resource \
  --main-version="a2a8ce"
```

作業成功後，您應該會看到以下輸出內容：

```
✓ WASM Plugin [my-wasm-plugin-resource] is now serving version "a2a8ce"
```

由於每次建立新的 Wasm 外掛程式資源時，Media CDN 都會儲存 Docker 映像檔摘要，因此回溯作業會使用上次推出前執行的程式碼版本。

```
gcloud alpha service-extensions wasm-plugins describe my-wasm-plugin-resource \
  --expand-config
```

以版本「a2a8ce」為例，該版本具有摘要 sha256:08c12...：

```
name: "my-wasm-plugin-resource"
mainVersion: "a2a8ce"
mainVersionDetails:
  image: "<LOCATION>-docker.pkg.dev/<PROJECT>/<REPOSITORY>/my-wasm-plugin"
  imageDigest: "<LOCATION>-docker.pkg.dev/<PROJECT>/<REPOSITORY>/my-wasm-
plugin@sha256:08c121dd7fd1e4d3a116a28300e9fc1fa41b2e9775620ebf3d96cb7119bd9976"
```

刪除 WasmAction 和 WasmPlugin

如要刪除 WasmAction、WasmPlugin 和相關聯的 WasmPluginVersions，請按照下列步驟操作。

首先，請從 Media CDN EdgeCacheService 設定中移除 WasmAction 的參照。

要**移除**的參考線：

```
wasmAction: "my-wasm-action-resource"
```

接著，更新編輯後的 EdgeCacheService 設定。

```
gcloud alpha edge-cache services import prod-media-service --source=my-service.yaml
```

接著，將 WasmPlugin 的主要版本更新為空白字串「`""`」。

```
gcloud alpha service-extensions wasm-plugins update my-wasm-plugin-resource --main-version=
```

```
""
```

最後，請依序執行下列刪除步驟。

```
gcloud alpha service-extensions wasm-actions delete my-wasm-action-resource
```

```
gcloud alpha service-extensions wasm-plugin-versions delete my-version \ --wasm-plugin=my-wasm-plugin-resource
```

```
gcloud alpha service-extensions wasm-plugins delete my-wasm-plugin-resource
```

11. 清理實驗室環境

完成程式碼研究室後，請記得清理實驗室資源，否則這些資源會持續運作並產生費用。

下列指令會刪除 Media CDN EdgeCache 服務、EdgeCache 設定和 Service Extensions 外掛程式。請依序執行下列刪除步驟。

```
gcloud edge-cache services delete media-cdn

gcloud edge-cache origins delete cloud-storage-origin

gcloud alpha service-extensions wasm-actions delete my-wasm-action-resource

gcloud alpha service-extensions wasm-plugins update my-wasm-plugin-resource --main-version=""

gcloud alpha service-extensions wasm-plugin-versions delete my-version-1 --wasm-plugin=my-wasm-plugin-resource

gcloud alpha service-extensions wasm-plugins delete my-wasm-plugin-resource

gcloud artifacts repositories delete service-extension-$PROJECT_ID --location=$LOCATION
```

上述每個指令都會要求您確認是否要刪除資源。

12. 恭喜！

恭喜，您已完成 Media CDN 的 Service Extensions 程式碼研究室！

涵蓋內容

- 如何設定 Media CDN，並將 Cloud Storage bucket 設為來源
- 如何建立具有自訂 HTTP 驗證的 Service Extension 外掛程式，並將其與 Media CDN 建立關聯
- 如何驗證 Service Extensions 外掛程式是否正常運作
- (選用) 如何管理 Service Extensions 外掛程式，例如更新、參照、回溯及刪除特定外掛程式版本

後續步驟

查看一些程式碼研究室...

- [為網頁應用程式新增推播通知](#)
- [使用 Workbox 建構 PWA](#)

其他資訊

- [高效能的 Service Worker 載入](#)
- [根據要求類型採用的 Service Worker 快取策略](#)

參考文件

- [Web App Manifest 說明文件](#)
 - [網頁應用程式資訊清單屬性 \(MDN\)](#)
 - [安裝並新增至主畫面](#)
 - [Service Worker 總覽](#)
 - [Service Worker 生命週期](#)
 - [高效能的 Service Worker 載入](#)
 - [離線教戰手冊](#)
-